IsiZulu AI-Detection Classifier — Training Methodology

Base model, dataset composition, training configuration, and end-to-end pipeline | 26 May 2026

AfroXLMR-Large Base model	**559.9 M** Model parameters	**20 390** Total labelled samples
16 377 Training samples	**4 epochs** Actual training run	**~22 min** Wall-clock time (RTX 5090)

1. Task Definition

The task is **binary text classification** for the IsiZulu language: given a passage of text, determine whether it was written by a **human** (label 0) or generated by an **AI / machine** (label 1). The classifier is intended to detect AI-generated IsiZulu content in contexts such as academic integrity, content moderation, and linguistic research.

> IsiZulu is a Bantu language spoken by approximately 12 million first-language speakers in South Africa. It is a low-resource language for NLP: most general-purpose language models have very limited IsiZulu training data, which makes specialised fine-tuning on an African-language-aware base model essential.

2. Base Model — AfroXLMR-Large-29L

The base model is **AfroXLMR-Large-29L**, an XLM-RoBERTa Large variant that was continually pre-trained on a corpus of African languages. It was loaded from a local path (`base_model/`) and a randomly-initialised two-layer classification head was attached on top of the encoder's pooled `[CLS]` representation.

Why AfroXLMR?

Standard XLM-RoBERTa was pre-trained predominantly on European and high-resource Asian languages. AfroXLMR extends this with dedicated pre-training on African language corpora, giving the model a richer vocabulary coverage and better subword tokenisation for IsiZulu morphology — a critical advantage for a morphologically complex agglutinative language.

> **Classifier head architecture:** `classifier.dense` (1024 → 1024, GELU, dropout 0.1) → `classifier.out_proj` (1024 → 2). Randomly initialised, trained from scratch during fine-tuning.

Property	Value
Architecture	XLM-RoBERTa (encoder-only)
Variant	AfroXLMR-Large-29L
Hidden size	1 024
Transformer layers	24
Attention heads	16
Intermediate size (FFN)	4 096
Total parameters	559.9 M
Vocabulary size	250 002 tokens
Max position embeddings	514 tokens
Attention dropout	0.1
Hidden dropout	0.1
Activation function	GELU
Model size on disk	~2.24 GB (`model.safetensors`)
Transformers version	4.47.1

3. Dataset

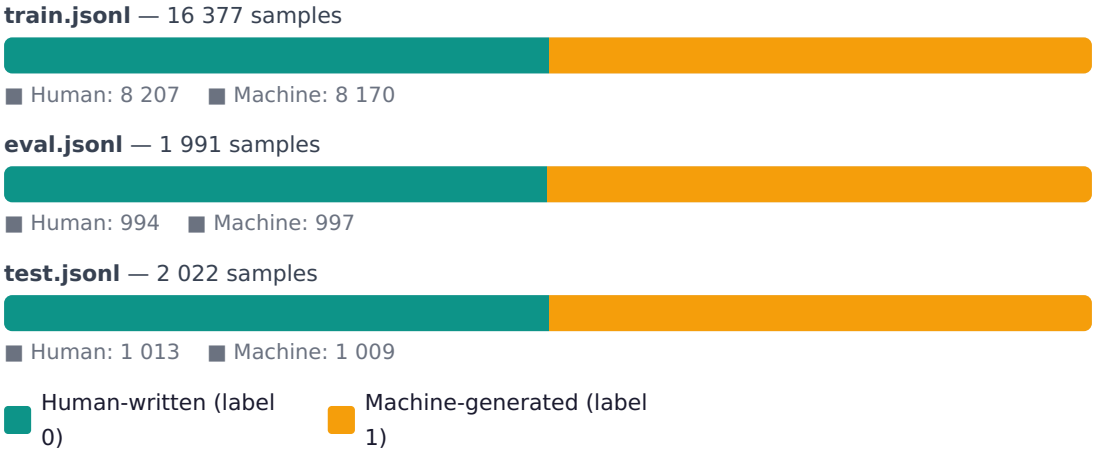
3.1 Format

All dataset files are in **JSONL** format — one JSON object per line. Each record contains two fields:

```
// Label 0 = human-written {"text": "Umculo udlala indima ebalulekile emphakathini wethu...", "label": 0} // Label 1 = machine-generated {"text": "IsiZulu yilulimi olukhulunywayo eNingizimu Afrika...", "label": 1}
```

3.2 Training & Evaluation Splits

Split	File	Total records	Human (label 0)	Machine (label 1)	Balance	Used for
Train	`train.jsonl`	**16 377**	**8 207 (50.1%)**	**8 170 (49.9%)**	**Near-perfect**	**Model training**
Validation	`eval.jsonl`	1 991	994 (49.9%)	997 (50.1%)	Near-perfect	Checkpoint selection / early stopping
Test Set 1	`test.jsonl`	2 022	1 013 (50.1%)	1 009 (49.9%)	Near-perfect	Held-out final evaluation



3.3 Additional Test Sets

File	Total records	Human (label 0)	Machine (label 1)	Distribution	Purpose
`test2.jsonl`	16 924	16 824 (99.4%)	100 (0.6%)	**Highly imbalanced**	Real-world distribution stress test
`test3.jsonl`	224	124 (55.4%)	100 (44.6%)	**Near-balanced**	Small held-out evaluation set

> **Total labelled data across all files:** 37 538 records (train + eval + test + test2 + test3). The training pipeline only uses `train.jsonl` (16 377) and `eval.jsonl` (1

991); the remaining 19 170 records across the three test files are held out entirely and never seen by the model during training.

4. Training Configuration

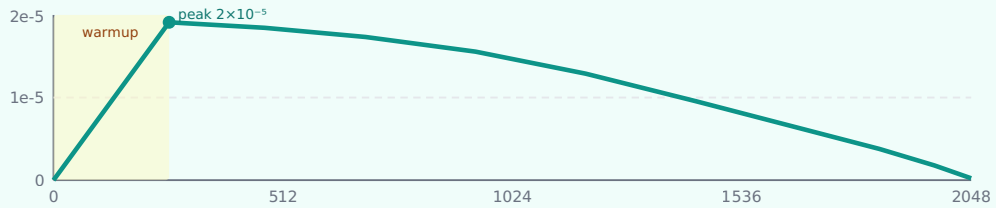
Hyperparameter	Value
Max sequence length	512 tokens
Batch size (per device)	8
Gradient accumulation steps	4
Effective batch size	**32 (8 × 4)**
Learning rate	2×10^{-5}
LR scheduler	Cosine decay
Warmup ratio	0.1 (first 10% of steps)
Weight decay	0.01
Max epochs	5
Early stopping patience	2 epochs
Best model metric	Validation F1 (machine class)
Mixed precision	FP16 (auto-enabled on CUDA)
Gradient checkpointing	Enabled
Evaluation strategy	Every epoch
Save strategy	Every epoch
Random seed	42

Infrastructure	Value
GPU	NVIDIA GeForce RTX 5090
VRAM	33.7 GB
CUDA version	12.1.1
cuDNN	8
PyTorch	2.3.1 (cu121)
Transformers	4.47.1
Datasets	4.8.5
Accelerate	1.13.0
Scikit-learn	1.7.2
Container base	nvidia/cuda:12.1.1-cudnn8-devel-ubuntu22.04
Total FLOPs	6.10×10^{16}
Throughput	186.6 samples/sec · 5.83 steps/sec
Wall-clock training time	~22 minutes (1 334 s)
Actual epochs completed	4 (early stop)
Total steps	2 048

Learning Rate Schedule

A **cosine decay** schedule was used with a linear warmup over the first 10% of training steps (~205 steps). The learning rate ramps from near-zero to the peak of 2×10^{-5}, then decays following a cosine curve back toward zero. This avoids catastrophic forgetting of the pre-trained representations during the early updates.

Learning rate over training steps (approximate curve)



5. Training Pipeline — Step by Step

STEP 1
Load Base Model
AfroXLMR-Large from local path. Attach random classifier head (1024 → 2).

→

STEP 2
Tokenize
XLMRobertaTokenizer. Max 512 tokens, pad to max_length, truncate. Batched map.

→

STEP 3
Build Subsets
Random 2 000-sample train subset for overfitting monitoring alongside eval set.

→

STEP 4
Train (Trainer API)
HuggingFace Trainer. FP16, grad checkpointing, grad accum 4. Eval every epoch.

→

STEP 5
Early Stopping
Monitor eval F1. Patience = 2. Best checkpoint auto-restored at end.

→

STEP 6
Save & Evaluate
Best model → final_model/. Full metrics on eval set and train subset saved to metrics.json.

6. Tokenisation & Input Preparation

Each text sample is tokenised using the **XLMRobertaTokenizer** (SentencePiece-based, vocabulary of 250 002 tokens). The tokenizer was loaded directly from the base model directory to ensure consistency with the pre-training vocabulary.

All sequences are padded or truncated to a fixed length of **512 tokens** — the model's maximum supported context window. Padding uses the <pad> token (ID 1); attention masks are set to 0 for padding positions so they do not influence the self-attention computation.

Token	ID	Role
<s>	0	Sequence start (CLS equivalent)
<pad>	1	Padding token
</s>	2	Sequence end (SEP equivalent)
<unk>	3	Unknown token
<mask>	250001	Masked language model token

> **Classification signal:** The representation of the first token (<s>, equivalent to [CLS]) after the final transformer layer is passed through the classifier head to produce the two logits (human / machine). This is standard practice for sentence-level classification with RoBERTa-family models.

Tokenisation setting	Value
Tokenizer class	XLMRobertaTokenizer
Max length	512
Padding	max_length
Truncation	True
Return tensors	PyTorch (pt)
Fields returned	input_ids, attention_mask
Batched mapping	Yes (HuggingFace datasets)

7. Evaluation Metrics

At the end of each epoch, the model is evaluated on two sets: the **validation set** (eval.jsonl, full 1 991 samples) and a **2 000-sample random subset of the training set** (for overfitting monitoring). The following metrics are computed for both:

Metric	Computed as
Accuracy	Fraction correctly classified
F1 (binary, machine)	Harmonic mean of precision & recall for label 1
Precision (machine)	TP / (TP + FP)
Recall (machine)	TP / (TP + FN)
F1 (human)	F1 score treating label 0 as positive
Precision (human)	TN / (TN + FN)
Recall (human)	TN / (TN + FP)
ROC-AUC	Area under ROC curve (probability scores)
MCC	Matthews Correlation Coefficient
TP / TN / FP / FN	Raw confusion matrix counts

Best model selection: The checkpoint is selected on `eval_eval_f1` — the binary F1 score on the validation set treating the *machine* class (label 1) as positive. This prioritises detecting AI-generated text, which is the core objective of the classifier. `load_best_model_at_end=True` ensures the Trainer automatically restores the best checkpoint at the end of training.

Softmax probabilities are extracted from the raw logits at evaluation time (numerically stable via log-sum-exp) to compute ROC-AUC, enabling threshold-independent assessment of the model's discriminative ability.

8. Reproducibility & Environment

Docker setup

Training is containerised via Docker Compose. Large artefacts (base model weights, datasets, output) are mounted as volumes at runtime — the image itself contains only code and dependencies, keeping it small and portable.

Volume	Mount point	Access
`./base model/`	`/app/base model/`	Read-only
`./datasets/`	`/app/datasets/`	Read-only
`./finetuned_model/`	`/app/finetuned_model/`	Read-write

Key dependencies

Package	Version
PyTorch	2.3.1 (cu121)
transformers	4.47.1
datasets	4.8.5
accelerate	1.13.0
scikit-learn	1.7.2
numpy	≥ 1.24
CUDA	12.1.1
cuDNN	8

Seed: 42 set via `TrainingArguments(seed=42)`, fixing dataset shuffling, weight initialisation of the classifier head, and dropout masks for full reproducibility.